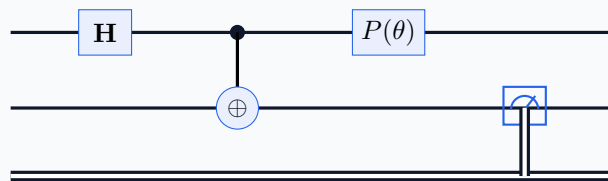


QPU Examples and Troubleshooting

Runnable Protocols, Diagnostics, and Correction Lab



Bailey Forbes

September 13, 2026

Complete examples with line-by-line explanations, advanced worked circuits (adders, comparators, parity, phase kickback, and oracles), custom gates, common protocol failures, diagnostic messages, truth-table checks, and guided Correction Lab recovery.

Contents

1	How to use the QPU website	1
1.1	Five programming words used in this guide	1
1.2	Interface words and how they relate	1
1.3	Run a bundled example without writing code	1
1.4	Compile a complete protocol	1
1.5	Complete examples versus fragments	2
1.6	When protocol checks report a problem	2
1.7	Saving and reopening work	2
2	First circuit: from source to result	2
2.1	Read the program line by line	2
2.2	What one run means	3
2.3	Adding user-controlled input	3
3	Worked example: reversible half-adder	3
3.1	Classical specification	3
3.2	QPU protocol	4
3.3	What every term is doing	4
3.4	Build the Carry gate in the workbench	4
3.5	Reversible state equation	5
3.6	Companion truth-table metadata	5
4	Worked example: randomness versus interference	5
4.1	Classical fair coin model	5
4.2	Quantum preparation with Hadamard	5
4.3	Insert phase and recombine paths	6
4.4	Term-by-term explanation	6
4.5	Comparison table	7
5	Advanced worked circuits	7
5.1	Full adder from primitive gates	7
5.2	Two-bit adder built from a child process	8
5.3	Reversible comparator: is A greater than B?	8
5.4	Two-bit equality with uncomputation	9
5.5	Parity of four bits	9
5.6	Phase kickback	10
5.7	A small oracle: finding a hidden bit string	10
5.8	Two-qubit Grover search	11
6	Process composition and return values	11
6.1	MAIN-PROCESS (lowered/executed)	12
6.2	DECLARECHILD (lowered/executed)	12

6.3	RUNCHILD — expand a child process (lowered/executed)	12
6.4	CALL — alternate child invocation (lowered/executed)	12
6.5	RETURNVALS (lowered/executed)	12
6.6	ACCEPTVALS (lowered/executed)	13
6.7	Nested full-adder example	13
6.8	Bounded child-process recursion (lowered/executed)	14
6.8.1	Recursion on the circuit canvas	15
6.9	Structured classical branches: IF / ELSE / ENDIF (lowered/executed)	15
6.9.1	Gate-expression conditions: IF (GATE ...) = V	16
6.9.2	IF / ELSE on the circuit canvas	16
6.10	Worked example: recursion, measurement, and IF / ELSE together	17
7	Custom gates: packaging a circuit as one block	17
7.1	Register one in four steps	18
7.2	How the process maps onto wires	18
7.3	Reasoning about reversibility	18
7.4	Using a custom gate in protocol text and IF / ELSE	19
7.5	Rules to remember	19
8	Truth-table companion syntax	20
8.1	Text .qpuio structure	20
8.2	MAIN-PROCESS: table header	20
8.3	INPUTS: and OUTPUTS: column groups	20
8.4	Column header row	20
8.5	Indexed data rows	20
8.6	CSV spelling	21
8.7	Partial truth tables	21
8.8	JSON .qpuio envelope	21
8.9	How a table test runs	22
8.10	Protected bundled tables	22
9	Complete worked protocol	22
10	Error conditions and practical rules	23
11	Troubleshooting protocols	23
11.1	Unknown instruction	23
11.2	Missing gate output	23
11.3	Wrong number of controls	23
11.4	Invalid phase angle	24
11.5	Unknown child process	24
11.6	Truth-table columns do not match	24
11.7	Cycle-suffix warnings	24
11.8	Using the Correction Lab	24

12 Diagnostic message reference	25
13 Glossary	25

1 How to use the QPU website

This chapter assumes no programming experience. You can build circuits by clicking controls, or edit the text protocol directly. Both paths describe the same circuit.

1.1 Five programming words used in this guide

Source text The human-readable instructions in the protocol editor. It is ordinary text, like a note, with one QPU instruction per logical line.

Instruction One action for the QPU system, such as SET, H, CNOT, or MEASURE. The first word says what action to perform.

Argument Extra information after an instruction. In SET Q 0p, Q and 0p are arguments: they say which wire and which state.

Compile Translate source text into the ordered visual gates that the simulator can run. Compilation checks spelling, required inputs, and targets.

Run Apply the compiled gates to the selected starting state. Running changes simulator state; it does not rewrite the source text.

1.2 Interface words and how they relate

Word on screen or in source	Meaning
Particle / qubit	One simulated quantum two-state system. The interface may say particle where circuit theory says qubit.
Wire	The horizontal visual path followed by one qubit through the circuit.
Token	A source-text name, such as Q or Carry, that resolves to a wire.
Register	One or more qubit wires considered as a group.
Gate	A box or controlled symbol that transforms state.
Target	The wire changed by a gate.
Control	A wire whose state decides whether a controlled gate acts.
Protocol / process	The complete source description of a named circuit.

1.3 Run a bundled example without writing code

1. Open the navigation menu and choose **Circuit builder**.
2. Find **Load a starter circuit** and choose an example.
3. Inspect the horizontal wires and gate blocks on the circuit canvas.
4. Choose **Play Sequence** to watch gates advance, or **Run all** to jump to the final state.
5. Choose **Measure all** if the circuit has not already measured its outputs.
6. Read the particle/state display and runtime log. A measured qubit is a classical 0 or 1 outcome for that run.
7. Choose **Reset state** before repeating the same experiment.

1.4 Compile a complete protocol

Listings marked **Complete runnable protocol** may be copied as a whole:

Complete runnable protocol

```
MAIN-PROCESS FirstWebsiteCircuit
CREATETOKEN -I Q
SET Q 0p
H -I Q -O Q
MEASURE -I Q
RETURNVALS Q
```

1. Scroll to **Compile text into a circuit**.
2. Replace the editor contents with the complete listing.
3. Choose **Compile protocol**.
4. Read **Protocol checks**. Zero errors means compilation may proceed. Warnings identify accepted syntax that may not behave as expected.
5. Return to the run controls and choose **Run all**.

Expected result: the H gate prepares equal 0 and 1 amplitudes. MEASURE returns either 0 or 1. Repeating from Reset state should produce both outcomes over many runs.

1.5 Complete examples versus fragments

This documentation uses three listing labels:

Complete runnable protocol Copy the entire block into the protocol editor and compile it.

Instruction fragment A few lines demonstrating one syntax rule. Add them to a complete process rather than compiling the fragment alone.

Companion metadata Truth-table text saved separately as `.qpuio`; do not paste it into the protocol editor.

1.6 When protocol checks report a problem

An **error** blocks successful compilation. A **warning** means the text is accepted but may use inactive, incomplete, or surprising behavior. Each diagnostic gives a line number, the relevant source line when available, and a suggested correction.

Choose **Open Correction Lab for guided help** below the diagnostic list when the correction is not obvious. The Correction Lab can test a protocol, probe outputs, explain failures, and propose gate-level guidance. It does not change protected bundled truth tables.

1.7 Saving and reopening work

Choose **Download as .qpucir** to save protocol source. The `-qpucir.txt` option contains the same source for devices that only handle plain text files. Use the navigation menu's **Upload files** page to reopen a protocol, optionally together with its matching `.qpuio` truth-table metadata.

2 First circuit: from source to result

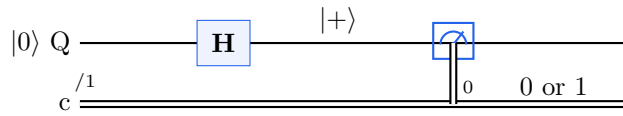
The shortest useful learning path is to prepare one qubit, transform it, measure it, and declare it as an output:

Complete runnable protocol

```
MAIN-PROCESS FirstCircuit
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

2.1 Read the program line by line

1. MAIN-PROCESS FirstCircuit gives the protocol a name.
2. CREATETOKEN -I Q reserves a logical wire named Q.
3. SET Q 0p prepares Q in the basis state $|0\rangle$.
4. H -I Q -0 Q applies Hadamard to that same wire, producing $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$.
5. MEASURE -I Q samples 0 or 1 and collapses the state.
6. RETURNVALS Q declares Q as the first public result.



2.2 What one run means

One execution gives one random-looking sample. For this circuit, theory predicts equal probability for 0 and 1. Getting 0 once does not mean H failed, and getting 1 once does not prove a bias. Repeat the circuit to compare observed frequencies with the predicted distribution.

2.3 Adding user-controlled input

Move a wire into PARAMS when callers or truth-table rows should choose its start state:

Complete runnable protocol

```
PARAMS: Input:state

MAIN-PROCESS Inverter
X -I Input -O Input
MEASURE -I Input
RETURNVALS Input
```

The matching deterministic truth table is:

Input	Output
0p	1p
1p	0p

The rest of this manual explains every symbol in these examples before moving to multi-qubit gates, child processes, and full truth-table files.

3 Worked example: reversible half-adder

A half-adder adds two one-bit numbers without a carry input. Classical theory defines two outputs:

$$\text{Sum} = A \oplus B, \quad \text{Carry} = A \wedge B.$$

The name *half-adder* means it handles half of the full-adder interface: there is no C_{in} input.

3.1 Classical specification

A	B	Carry	Sum	Binary interpretation
0	0	0	0	$0 + 0 = 00_2$
0	1	0	1	$0 + 1 = 01_2$
1	0	0	1	$1 + 0 = 01_2$
1	1	1	0	$1 + 1 = 10_2$

The Carry column is the high-order bit and Sum is the low-order bit. In the last row, decimal $1 + 1 = 2$, written 10_2 in binary.

3.2 QPU protocol

Complete runnable protocol

```

PARAMS: A:state B:state

MAIN-PROCESS HalfAdder
CREATETOKEN -I Carry Sum
SET Carry 0p
SET Sum 0p

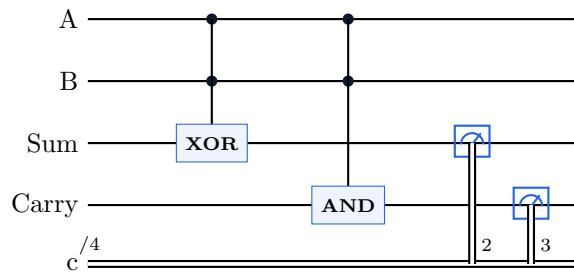
XOR -I A B -O Sum
AND -I A B -O Carry

MEASURE -I Carry
MEASURE -I Sum
RETURNVALS Carry Sum

```

3.3 What every term is doing

1. **PARAMS** declares A and B as caller-controlled state inputs. In a basis-state truth-table test they receive 0p or 1p.
2. **Carry** and **Sum** are **workspace targets**. CREATETOKEN reserves their names; SET prepares both in zero.
3. **XOR** uses **A** and **B** as **controls/inputs**. It mutates Sum according to $s' = s \oplus A \oplus B$. Because $s = 0$, the result is $A \oplus B$.
4. **AND** uses **the same preserved controls**. It mutates Carry according to $c' = c \oplus (A \wedge B)$. Because $c = 0$, Carry becomes the classical carry predicate.
5. **MEASURE** produces **classical samples**. For basis inputs this example is deterministic. If A or B is in superposition, outputs may be entangled with the inputs and measurement samples a branch.
6. **RETURNVALS** defines **interface order**. Carry is output column zero and Sum is output column one. It does not perform measurement itself.

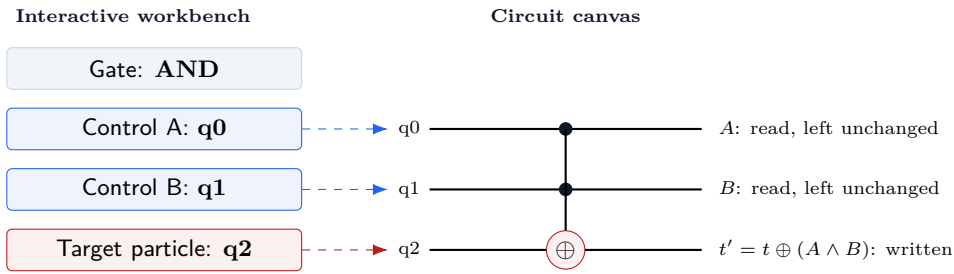


Which wires change?

Only the two outputs. A and B are *read* by both gates and come out exactly as they went in. Sum and Carry are *workspace*: they start at 0 and each receives one answer. Keeping the inputs is what makes the circuit reversible, and it is why a quantum half-adder needs four wires where a classical one draws two inputs and two outputs.

3.4 Build the Carry gate in the workbench

The AND line of the protocol corresponds to these workbench choices. With A on q0, B on q1, and Carry on q2:



Try it in the Circuit builder

Choose the *Half adder* starter circuit. Set **q0 start** and **q1 start** to each combination of 0p and 1p, choose **Run all** each time, and check the measured Sum and Carry against the table above.

3.5 Reversible state equation

Before measurement, the full basis-state action is

$$|A, B, s, c\rangle \mapsto |A, B, s \oplus A \oplus B, c \oplus (A \wedge B)\rangle.$$

The familiar half-adder is the special case $s = c = 0$. Writing the complete equation avoids a common mistake: XOR and AND do not overwrite arbitrary targets. They reversibly combine results with the targets' previous states.

3.6 Companion truth-table metadata

```
MAIN-PROCESS: HalfAdder
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
0 0p 0p 0p 0p
1 0p 1p 0p 1p
2 1p 0p 0p 1p
3 1p 1p 1p 0p
```

This file compares the measured QPU outputs with the classical half-adder specification. Passing all four rows demonstrates basis-state equivalence; it does not by itself characterize phases for superposition inputs.

4 Worked example: randomness versus interference

Classical random choice and quantum superposition can produce identical single-measurement frequencies. They are not the same theory. This example uses an intermediate phase change to distinguish them.

4.1 Classical fair coin model

Imagine a classical variable R chosen by a fair coin:

$$P(R = 0) = \frac{1}{2}, \quad P(R = 1) = \frac{1}{2}.$$

Once chosen, R is one definite value even if the observer has not looked. Applying classical NOT changes 0 to 1 and 1 to 0, but the distribution remains half and half. There is no operation that makes the probability of one hidden branch cancel the other branch.

4.2 Quantum preparation with Hadamard

Complete runnable protocol

```

MAIN-PROCESS QuantumCoin
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q

```

Before measurement,

$$|0\rangle \xrightarrow{H} |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

The amplitudes are $1/\sqrt{2}$ and $1/\sqrt{2}$; squaring magnitudes gives the same half-and-half distribution as the classical coin. If measurement happens now, this experiment alone cannot distinguish the two descriptions.

4.3 Insert phase and recombine paths

Complete runnable protocol

```

MAIN-PROCESS PhaseInterference
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
Z -I Q -0 Q
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q

```

Track the state:

$$|0\rangle \xrightarrow{H} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \xrightarrow{Z} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \xrightarrow{H} |1\rangle.$$

4.4 Term-by-term explanation

Superposition After the first H, both basis states have coherent amplitudes. No measurement has selected one.

Phase Z multiplies only the $|1\rangle$ amplitude by $-1 = e^{i\pi}$. Its magnitude remains $1/\sqrt{2}$, so immediate 0/1 probabilities remain equal.

Relative phase The plus relationship between amplitudes becomes a minus relationship. A global minus on both terms would not matter; changing only one term does.

Interference The final H mixes the basis amplitudes. Contributions to the $|0\rangle$ output cancel, while contributions to $|1\rangle$ add.

Measurement The final state is $|1\rangle$, so measurement returns 1 with probability one.

The cancellation can be seen directly from matrix multiplication:

$$H \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Classical probability weights are nonnegative and would add rather than cancel. Quantum amplitudes are signed or complex, so phase can create destructive and constructive interference before the Born rule turns amplitudes into probabilities.

4.5 Comparison table

Question	Classical fair coin	H–Z–H quantum circuit
State before observation	One hidden definite bit sampled fairly	Coherent complex amplitude vector
Effect of middle sign/phase	No probability analogue that preserves a fair distribution and later cancels a branch	Z changes relative phase without changing immediate basis probabilities
Can paths cancel?	No; probabilities add	Yes; amplitudes add and may cancel
Final result in this example	Still a distribution unless another classical rule forces a value	Deterministic 1

5 Advanced worked circuits

The earlier examples use one or two gates. The circuits in this chapter show that the same pieces scale to arithmetic, comparison, parity, phase tricks, and small oracle algorithms. Every listing is a **Complete runnable protocol**: paste it into **Compile text into a circuit**, choose **Compile protocol**, then **Run all**. Circuits with PARAMS can also be checked row by row in the **Circuit correction lab** with **Infer truth table**, **Probe outputs**, and **Test circuit**.

One pattern runs through this whole chapter

Inputs are *read*, never overwritten. Each answer is written into a fresh output wire that starts at 0, using the rule $\text{target}' = \text{target} \oplus f(\text{inputs})$. When a circuit needs a temporary helper wire, it computes the helper, uses it, and then runs the same gates again to return the helper to 0 (*uncomputation*).

5.1 Full adder from primitive gates

A full adder adds three bits A , B , and a carry-in C_{in} :

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = \text{majority}(A, B, C_{\text{in}}) = (A \wedge B) \oplus (A \wedge C_{\text{in}}) \oplus (B \wedge C_{\text{in}}).$$

Complete runnable protocol

PARAMS: A:state B:state Cin:state

MAIN-PROCESS FlatFullAdder

CREATETOKEN -I Cout Sum

SET Cout 0p

SET Sum 0p

Stage 1: Sum collects the parity of the three inputs.

CNOT -I A -O Sum

CNOT -I B -O Sum

CNOT -I Cin -O Sum

Stage 2: Cout flips once for every pair of inputs that are both 1.

CCNOT -I A B -O Cout

CCNOT -I A Cin -O Cout

CCNOT -I B Cin -O Cout

MEASURE -I Cout

MEASURE -I Sum

RETURNVALS Cout Sum

Why the carry works: if exactly two inputs are 1, one pair is active and Cout flips once, to 1. If all three are 1, all three pairs are active and Cout flips three times, which still leaves 1. With zero or one active input no pair fires. That is exactly “at least two of three”, the majority function. The bundled **SingleBitFullAdder** is the same circuit.

5.2 Two-bit adder built from a child process

Multi-bit addition chains full adders: the carry out of bit 0 becomes the carry in of bit 1. RUNCHILD reuses the bundled SingleBitFullAdder for each bit, so the parent only describes the wiring.

Complete runnable protocol

```
PARAMS: A1:state A0:state B1:state B0:state

MAIN-PROCESS TwoBitRippleAdder
DECLARECHILD SingleBitFullAdder
CREATETOKEN -I NoCarry C0 S0 C1 S1
SET NoCarry Op

# Bit 0: A0 + B0 with no incoming carry.
RUNCHILD SingleBitFullAdder -I A0 B0 NoCarry -O C0 S0
# Bit 1: A1 + B1 plus the carry from bit 0.
RUNCHILD SingleBitFullAdder -I A1 B1 C0 -O C1 S1

MEASURE -I C1
MEASURE -I S1
MEASURE -I S0
RETURNVALS C1 S1 S0
```

The outputs read as a three-bit binary number $C_1S_1S_0$. For example, $A = 11_2 = 3$ and $B = 10_2 = 2$ give $C_1S_1S_0 = 101_2 = 5$. The bundled TwoBitFullAdder and FourBitFullAdder extend the same chain with an explicit carry input and more bits.

5.3 Reversible comparator: is A greater than B?

For single bits, $A > B$ exactly when $A = 1$ and $B = 0$, that is $A \wedge \neg B$. The circuit negates B temporarily, uses it, and restores it.

Complete runnable protocol

```
PARAMS: A:state B:state

MAIN-PROCESS GreaterThan
CREATETOKEN -I Gt
SET Gt Op

X -I B -O B
AND -I A B -O Gt
X -I B -O B

MEASURE -I Gt
RETURNVALS Gt
```

A	B	Gt
0	0	0
0	1	0
1	0	1
1	1	0

The second X is what keeps the circuit honest: without it, B would leave the circuit inverted and any later gate reading B would see the wrong value.

5.4 Two-bit equality with uncomputation

Two numbers are equal when every bit agrees. This circuit writes a “this bit differs” flag for each bit, combines the flags, and then cleans the flags up.

Complete runnable protocol

```
PARAMS: A1:state A0:state B1:state B0:state

MAIN-PROCESS TwoBitEquality
CREATETOKEN -I D1 D0 Eq
SET D1 0p
SET D0 0p
SET Eq 1p

# Compute: D = 1 where the bits differ.
XOR -I A1 B1 -O D1
XOR -I A0 B0 -O D0

# Eq starts at 1 and flips if any bit differs.
OR -I D1 D0 -O Eq

# Uncompute: the same XORs return D1 and D0 to 0.
XOR -I A1 B1 -O D1
XOR -I A0 B0 -O D0

MEASURE -I Eq
MEASURE -I D1
MEASURE -I D0
RETURNVALS Eq D1 D0
```

Three ideas meet here:

A target that starts at 1 SET Eq 1p makes $\text{Eq}' = 1 \oplus (D_1 \vee D_0) = \neg(D_1 \vee D_0)$. Starting a target at 1 turns “flip when true” into “clear when true”.

Uncomputation XOR is its own inverse, so repeating the two XOR lines returns D_1 and D_0 to 0 for every input.

Proof by truth table Returning D1 and D0 lets a truth-table test confirm the cleanup. In all 16 rows those two columns must be 0.

5.5 Parity of four bits

Parity is 1 when an odd number of inputs are 1. Each CNOT adds one input into the target modulo 2.

Complete runnable protocol

```
PARAMS: A:state B:state C:state D:state

MAIN-PROCESS FourBitParity
CREATETOKEN -I P
SET P 0p

CNOT -I A -O P
CNOT -I B -O P
CNOT -I C -O P
CNOT -I D -O P

MEASURE -I P
RETURNVALS P
```

Parity checks are how error-detecting codes notice a single flipped bit. The two-input XOR -I A B -O P is shorthand for the first two CNOT lines.

5.6 Phase kickback

A CNOT never changes its control's 0/1 value, but it *can* change the control's phase. Put the target in $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$: flipping $|-\rangle$ gives $-|-\rangle$, so “flip the target” becomes “multiply by -1 ”, and that sign attaches to the control's $|1\rangle$ branch.

Complete runnable protocol

```
MAIN-PROCESS PhaseKickback
CREATETOKEN -I Ctrl Tgt
SET Ctrl 0p
SET Tgt 1p

H -I Ctrl -0 Ctrl
H -I Tgt -0 Tgt
CNOT -I Ctrl -0 Tgt
H -I Ctrl -0 Ctrl

MEASURE -I Ctrl
RETURNVALS Ctrl
```

$$|+\rangle|-\rangle \xrightarrow{\text{CNOT}} |-\rangle|-\rangle \xrightarrow{H \text{ on Ctrl}} |1\rangle|-\rangle.$$

Ctrl measures 1 every time. Change `SET Tgt 1p` to `SET Tgt 0p` and it measures 0 every time: the target's state decided the control's result, even though the control was never “written”. The *Phase kickback* starter circuit shows the same effect on the canvas.

5.7 A small oracle: finding a hidden bit string

An *oracle* is a circuit treated as a black box that computes some $f(x)$. Here $f(x) = s \cdot x \bmod 2$ for a hidden three-bit string s (Bernstein–Vazirani). Classically, finding s takes three queries. Using kickback, one query is enough.

Complete runnable protocol

```
MAIN-PROCESS HiddenString101
CREATETOKEN -I In2 In1 In0 Anc
SET In2 0p
SET In1 0p
SET In0 0p
SET Anc 1p

H -I In2 -0 In2
H -I In1 -0 In1
H -I In0 -0 In0
H -I Anc -0 Anc

# Oracle for s = 101: one CNOT for every 1 in s.
CNOT -I In2 -0 Anc
CNOT -I In0 -0 Anc

H -I In2 -0 In2
H -I In1 -0 In1
H -I In0 -0 In0

MEASURE -I In2
MEASURE -I In1
MEASURE -I In0
RETURNVALS In2 In1 In0
```

The measured outputs are 1,0,1: the hidden string itself. Each oracle CNOT kicks a phase back onto one input wire, and the final H gates turn those phases into 0/1 values. Change which inputs have an oracle CNOT and the answer changes to match. With a single input wire this is Deutsch's algorithm.

5.8 Two-qubit Grover search

Grover's algorithm finds a marked input by boosting its amplitude. With two qubits, one round finds the answer with certainty.

Complete runnable protocol

```

MAIN-PROCESS GroverFind11
CREATETOKEN -I A B
SET A 0p
SET B 0p

# Equal superposition of 00, 01, 10, 11.
H -I A -0 A
H -I B -0 B

# Oracle: mark 11 with a minus sign.
CZ -I A -0 B

# Diffusion: reflect every amplitude about the average.
H -I A -0 A
H -I B -0 B
X -I A -0 A
X -I B -0 B
CZ -I A -0 B
X -I A -0 A
X -I B -0 B
H -I A -0 A
H -I B -0 B

MEASURE -I A
MEASURE -I B
RETURNVALS A B

```

Amplitude of	$ 00\rangle$	$ 01\rangle$	$ 10\rangle$	$ 11\rangle$
After H, H	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$
After oracle	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$-\frac{1}{2}$
After diffusion	0	0	0	± 1

The oracle does not change any probability, since every entry still has magnitude $\frac{1}{2}$. The diffusion stage turns the marked sign into certainty. Open **Outcome truth table / probabilities** on the **Particle visualization** page after each **Step gate** to watch the amplitudes move.

6 Process composition and return values

Main process and child process in plain words

A **process** is a named list of gates with inputs (**PARAMS**) and outputs (**RETURNVALS**). The **main process** is the one you compile and run: the name after **MAIN-PROCESS**. A **child process** is another saved process that the main process borrows with **RUNCHILD**, the way a program calls a helper function.

The difference from a function call is that nothing is called at run time. The compiler copies the child's gates into the main circuit, on the wires you pass: **-I** wires feed the child's inputs in **PARAMS** order, and **-O** wires receive its outputs in **RETURNVALS** order. The canvas therefore shows every gate, including the ones that came from children.

"Main" and "child" are roles in one compile, not kinds of file. **SingleBitFullAdder** is the main process when you run it on its own and a child when **TwoBitFullAdder** runs it twice. A custom gate is the canvas version of the same idea: a saved process placed on wires you choose.

Browse a process and its children

In the Circuit builder, press the **TwoBitFullAdder** protocol button. The Interactive workbench gains a **Compiled protocol** card showing the row of its truth table that matches the current start states. Open **Learn more** on that card for the full 32-row table (grouped by the leading input bits), its notes, and a **SingleBitFullAdder** entry you can expand in place to read the child.

6.1 MAIN-PROCESS (lowered/executed)

```
MAIN-PROCESS FullAdder
```

This marks the process body and supplies its catalog/library name. Compilation already begins from the named process, so the row itself emits no gate. The name is the first positional word after **MAIN-PROCESS**; avoid spaces inside it. This name connects a protocol to catalog entries, child calls, downloads, and companion truth-table metadata. If no marker exists, the system uses **InlineProcess**, but named processes are strongly recommended.

6.2 DECLARECHILD (lowered/executed)

```
DECLARECHILD SingleBitFullAdder
```

The named child is bound from the compilation **librarySources** catalog. An unknown name fails on this line. The row itself emits no gate; later **RUNCHILD** or **CALL** expansions use the bound process body. A successful declaration therefore guarantees that the named child exists in the catalog supplied to this compile, and that later calls of that name expand the bound body rather than a later alias.

6.3 RUNCHILD — expand a child process (lowered/executed)

```
RUNCHILD SingleBitFullAdder \
  -I A B Cin \
  -O Carry Sum
```

The first positional argument is the child process name. Inputs bind by the child's **PARAMS** order: **A** maps to the first parameter, **B** to the second, and **Cin** to the third in this example. Outputs bind by the child's **RETURNVALS** order. Before expansion, each bound parent output is zero-prepared once. Child-local names receive a unique scope.

RUNCHILD is compile-time composition, not a dynamic subroutine call in a running quantum machine. The child body is recursively expanded into the parent's flat gate sequence. This allows the renderer and simulator to inspect every resulting gate.

6.4 CALL — alternate child invocation (lowered/executed)

```
CALL SingleBitFullAdder -I A B Cin -O Carry Sum
```

CALL currently follows the same child lookup, parameter binding, output preparation, and expansion path as **RUNCHILD**. **RUNCHILD** is used by the bundled multi-bit protocols and makes compile-time expansion explicit; **CALL** is retained as an equivalent system spelling. Both fail when the named child is absent from the supplied process library.

6.5 RETURNVALS (lowered/executed)

```
RETURNVALS Carry Sum
```

Positional tokens define the process's ordered public outputs. They determine child output binding, truth-table output columns, and the UI's preferred logical width. Do not use **-I** or **-O** here: flags would be retained as positional return tokens.

RETURNVALS does not measure automatically. Add MEASURE instructions when classical outcomes are required. Returning an unmeasured wire preserves it as part of the projected quantum output shown by the system.

6.6 ACCEPTVALS (lowered/executed)

```
CALL Child -I A B
ACCEPTVALS LocalCarry LocalSum
```

Each positional local token aliases the corresponding value from the most recent child return list. Direct `-0` binding on RUNCHILD/CALL is clearer when output names are known.

Ordering is again essential: the first ACCEPTVALS name receives the first returned reference, and so on. The instruction refers to the most recently completed child expansion in compiler state; inserting another child call changes which return list is accepted.

6.7 Nested full-adder example

```
# child.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS ChildAdder
CREATETOKEN -I Carry Sum
SET Carry Op
SET Sum Op
XOR -I A B -O Sum
CNOT -I Cin -O Sum
CCNOT -I A B -O Carry
CCNOT -I A Cin -O Carry
CCNOT -I B Cin -O Carry
RETURNVALS Carry Sum

# parent.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS Parent
CREATETOKEN -I Cout Result
RUNCHILD ChildAdder -I A B Cin -O Cout Result
MEASURE -I Cout
MEASURE -I Result
RETURNVALS Cout Result
```

For basis inputs, the full-adder equations are

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = (A \wedge B) \oplus (A \wedge C_{\text{in}}) \oplus (B \wedge C_{\text{in}}).$$

<i>A</i>	<i>B</i>	<i>C_{in}</i>	<i>C_{out}</i>	<i>Sum</i>
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

6.8 Bounded child-process recursion (lowered/executed)

The QPU protocol does not support runtime loops or recursive execution of the compilation root. A child marked REC may recursively expand itself during compilation only. The parent supplies a finite `-DEPTH`; each RECUR creates another compile-time frame with `DEPTH - 1`. `EXIT WHEN ...` is the compile-time base case. After expansion the simulator still receives a flat ordered gate list.

```
# recursive-h.qpucir
PARAMS: Q:state
MAIN-PROCESS RecursiveH
REC MAXDEPTH 16
EXIT WHEN DEPTH == 0
H -I Q -0 Q
INCREASECYCLE
RECUR -I Q
RETURNVALS Q
```

```
# parent
PARAMS: Q:state
MAIN-PROCESS Parent
DECLARECHILD RecursiveH
RUNCHILD RecursiveH -DEPTH 4 -I Q
RETURNVALS Q
```

Compiler structure:

Parent $\rightarrow$ RecursiveH DEPTH 4 $\rightarrow$ DEPTH 3 $\rightarrow$ DEPTH 2 $\rightarrow$ DEPTH 1 $\rightarrow$ stop

Emitted circuit (no recursion metadata remains):

```
H
INCREASECYCLE
H
INCREASECYCLE
H
INCREASECYCLE
H
INCREASECYCLE
```

Rules (summary):

- Root $\rightarrow$ root is forbidden.
- Child $\rightarrow$ same child is allowed when bounded.
- Mutual recursion ($A \rightarrow B \rightarrow A$) is rejected.
- RECUR requires REC or TREC; the initial RUNCHILD requires `-DEPTH`.
- DEPTH, LEVEL, and ROOTDEPTH are read-only.
- Absolute compiler limit: 64; optional REC/TREC MAXDEPTH n is lower.
- In-place `-I` (same wire returned) is not zero-prepared between recursive frames.
- **TCO (F#-style)**: if every RECUR is in tail position, REC automatically expands iteratively in one compile-time frame (looping rewind). Explicit TREC asserts the same form and errors if post-RECUR work exists. Non-tail REC keeps stacked frames.

Keep these concepts distinct:

Construct	Meaning
RUNCHILD	Compose another process
RECUR	Expand this child again with $\text{DEPTH}-1$
REC	Allow recursion; auto-TCO when tail
TREC	Require tail form; always TCO
INCREASECYCLE	Next logical circuit stage (not a loop)
WHEN	Compile-time condition on $\text{DEPTH}/\text{LEVEL}/\text{ROOTDEPTH}$
-IF	Runtime classical feed-forward after MEASURE
IF/ELSE/ENDIF	Block form of -IF with an alternative branch

The existing bit adders remain the teaching example of explicit hierarchical composition. Recursion is a circuit-*construction* abstraction for repeated stages; it never introduces a backward execution edge.

6.8.1 Recursion on the circuit canvas

The compiled gate list holds every unrolled frame, but the canvas draws each recursive call once:

- A single-gate body (such as **RecursiveH**) keeps its own symbol. A body with several gates collapses to one **REC** box that spans every wire the body touches. Above the box, a caption gives the child name and the depth range, for example **RecursiveReversibleEcho**, $D5 \rightarrow D1$.
- A teal Dn badge above the gate shows the current DEPTH. It reads the root depth before the call, counts down ($D5, D4, \dots$) as **Step gate** walks through the frames, and hides once the call has finished.
- The REC box follows the usual colours: red while a forward gate of the frame is running, purple while an inverse (dg/inv) gate is running.
- The INCREASECYCLE markers inside the expansion are not drawn on the canvas; the badge replaces them.

While the playhead is inside a recursive call, the Interactive workbench shows a **Recursive frame** card. It draws the level being executed with the same gate symbols as the canvas: the heading gives $\text{DEPTH } d \cdot \text{LEVEL } \ell \cdot \text{TCO}$ (or **STACK**), the forward gates sit above a **forward** $\rightarrow$ rule and the adjoint gates above a $\leftarrow$ **inverse** rule, the frame's INCREASECYCLE is a dashed line labelled $\text{IC}n$, and the frame ends in **RECUR** $\downarrow D(d-1)$ (**D0 exit** on the last frame). The current gate is highlighted. When the playhead is elsewhere the card is collapsed as *Expand ... first frame*. The downward arrow is deliberate: the next frame is the next block of the finite circuit, not a jump back to an earlier column.

6.9 Structured classical branches: IF / ELSE / ENDIF (lowered/executed)

A block conditions every gate inside it on one measured bit, with an optional alternative:

```
MEASURE -I A
IF A=1
X -I C -O C
ELSE
Z -I C -O C
ENDIF
```

The compiler removes IF, ELSE, and ENDIF and emits ordinary gates. Each gate in the IF part carries the condition $A=1$; each gate in the ELSE part carries the opposite value, $A=0$. The example above compiles exactly as

```
MEASURE -I A
X -I C -O C -IF A=1
Z -I C -O C -IF A=0
```

plus canvas metadata that records which gates came from the IF part and which from the ELSE part.

Rules:

- The header is **IF Token=0** or **IF Token=1**, with no spaces around $=$. The token may be a parameter or any name that is in scope.

- **ELSE** is optional and takes no arguments. It flips the value for the gates after it. **ENDIF** is required and takes no arguments.
- **ELSE** or **ENDIF** without a matching **IF**, or an **IF** left open at the end of the process, is a compile error.
- The token must be measured before the first conditioned gate runs; otherwise the simulator stops with “requires *qn* to be measured first”.
- A gate carries exactly one condition, so these forms are compile errors rather than silently dropping a test: an **IF** inside another open **IF**; a gate with its own **-IF** inside a block; and **RUNCHILD**, **CALL**, or **RECUR** inside a block (the child’s gates would not inherit the condition). Close the block with **ENDIF** first, condition the child’s gates inside the child, or register the child as a custom gate and use that gate inside the block.
- Conditions inside a custom gate’s process are evaluated the same way when the custom gate runs.
- Both branches remain in the compiled forward sequence. At run time the gate whose condition fails is skipped (the run log says “skipped because classical condition was false”); nothing is removed and nothing forks.

6.9.1 Gate-expression conditions: **IF (GATE ...)** = V

Instead of one measured bit, the header can test the result of any gate. This is how conditions are joined: where another language writes **A && B**, QPU writes the **AND** gate.

```
IF (AND -I A B -O B) = 1p      # A AND B
IF (OR -I A B -O B) = 1       # A OR B
IF (CNOT -I A -O B) = 1       # A XOR B (B is the CNOT target)
IF (NAND -I A B -O B) != 0p   # != also accepted
IF (H -I A -O A) = S         # would H leave A in superposition?
```

The value after **=** or **!=** is **0/0p**, **1/1p**, or **S/sp** (any case).

How it is evaluated:

- Just before each conditioned gate runs, the simulator applies the expression gate to a *scratch copy* of the current state. The circuit’s own wires are not changed by the test.
- The gate is read exactly like a circuit line: **-O** names the result wire, and **-I** wires are its inputs. So **X -I A -O C** flips **C** and tests **C**; it does not copy **A**.
- One exception makes Boolean joins read naturally: when **AND**, **NAND**, **OR**, or **XOR** names one of its own inputs as **-O** (as in **AND -I A B -O B**), all inputs stay operands and the result is written to a fresh wire that starts at **|0>**. The test is therefore $A \wedge B$.
- The result wire is classified by its probability of reading 1: **0** (definitely **|0>**), **1** (definitely **|1>**), or **S** (anything in between, including entangled wires). **sp** means “superposed”, not only **|+>**.
- **ELSE** runs its gates when the test is false (for **= 1** that means **0** or **S**).
- The expression must be a single gate with one result wire. Built-in gates and registered custom gates with one **RETURNVALS** value both work; a custom gate keeps its own wiring and reads its **-O** wire. **MEASURE**, **RESET**, **SWAP**, and **RUNCHILD** are rejected, as is a **-IF** inside the expression. Blocks still cannot be nested.
- Custom gates can also sit inside the branch bodies, where the whole gate runs or is skipped as one step. See *Custom gates* for the syntax.

Simulator-only test

A measured-bit **IF A=1** is ordinary feed-forward and can run on hardware. A gate-expression test reads amplitudes without measuring, which a simulator can do but a real quantum processor cannot. Nothing collapses, and the **S** result in particular has no physical equivalent. Use it to explore and teach logic, and use **MEASURE** plus **IF Token=0|1** when a protocol must be physically realistic.

6.9.2 **IF** / **ELSE** on the circuit canvas

The quantum wire stays a single straight line. The branch gates are drawn one after another on that wire, never as a fork:

- A yellow double line drops from each conditioned gate to the classical **c** lane. It passes behind any gate it crosses.
- Under the **c** lane each branch is labelled with its keyword and test, for example **IF A=1** and **ELSE A=0**. Compiled protocols use the parameter name; canvas circuits use *qn*.
- Before the bit is measured both branches are drawn normally. After the measurement the branch that runs shows

- ✓ **taken**; the other shows ∅ **skipped**, and its gate, feed line, and label fade and turn dashed. If the measured value changes, the two swap.
- A gate-expression test has no bit on **c**. Its yellow double line instead spans the wires the expression reads, with a small yellow tap on each, and the label shows the expression, for example **IF AND(A,B) = 1**. Taken/skipped appears once the simulator has evaluated it.

6.10 Worked example: recursion, measurement, and IF / ELSE together

The bundled **RecursiveReversibleEchoHarness** protocol combines both features. Its child applies a forward block and then the adjoints of that block in reverse order, so each frame returns the wires to where they started:

```
# recursive-reversible-echo.qpucir (abridged)
PARAMS: A:state B:state C:state
MAIN-PROCESS RecursiveReversibleEcho
TREC MAXDEPTH 8
EXIT WHEN DEPTH == 0
H -I A -O A
RX=pi/3 -I B -O B
RY=pi/5 -I C -O C
CNOT -I A -O B
CPHASE=pi/7 -I B -O C
T -I A -O A
AND -I A B -O C
SWAP -I B C -O B C
SWAPdg -I B C -O B C
ANDinv -I A B -O C
Tdg -I A -O A
invCPHASE=pi/7 -I B -O C
CNOTdg -I A -O B
RYinv=pi/5 -I C -O C
dgRX=pi/3 -I B -O B
Hinv -I A -O A
INCREASECYCLE
RECUR -I A B C
RETURNVALS A B C
```

The harness prepares $|101\rangle$, runs the child with `-DEPTH 5`, and then branches on the measured value of A:

```
X -I A -O A
X -I C -O C
RUNCHILD RecursiveReversibleEcho -DEPTH 5 -I A B C -O A B C
ACCEPTVALS A B C
MEASURE -I A
IF A=1
X -I C -O C
ELSE
Z -I C -O C
ENDIF
MEASURE -I B
MEASURE -I C
```

Because every frame undoes itself, A is still 1 after the echo, so the **IF** branch runs: C flips to 0 and the final reading is A=1, B=0, C=0. On the canvas the call is one REC box whose badge counts $D5 \rightarrow D1$; the X on C shows ✓ **taken** and the Z shows ∅ **skipped**.

7 Custom gates: packaging a circuit as one block

A custom gate is a whole QPU process saved under a short name so that it can be placed like any built-in block. It adds no new physics: when it runs, the simulator replays the process's own gates, one by one, on the wires you chose.

Understanding a custom gate therefore means understanding the smaller gates inside it.

7.1 Register one in four steps

1. Paste the process below into the protocol editor in **Compile text into a circuit** and choose **Compile protocol** to confirm it has no errors.
2. In **Register custom gates from QPU processes**, type NOR in **Gate id (palette label)** and leave **Source process** on *Current protocol editor*.
3. Choose **Register custom gate**. A **NOR** block appears under **Custom gates** in the palette and in the workbench **Gate** list.
4. Place it with **Control A**, **Control B**, and **Target particle**, exactly like AND.

Complete runnable protocol

```
PARAMS: A:state B:state
```

```
MAIN-PROCESS NorGate
```

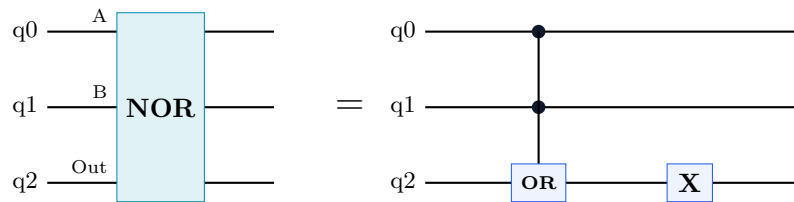
```
CREATETOKEN -I Out
```

```
OR -I A B -O Out
```

```
X -I Out -O Out
```

```
RETURNVALS Out
```

7.2 How the process maps onto wires



In the process	Becomes, when the block is placed
1st PARAMS entry	Control A (the lowest free wire when dropped on the canvas).
2nd PARAMS entry	Control B . Further entries take the next free wires.
1st RETURNVALS entry	Target particle : the only wire the block should change.
Any other token	A fresh extra workspace wire that the block allocates for itself.

The palette gives every custom block exactly one target wire, so register processes with a single **RETURNVALS** output. A process with two outputs, such as a half-adder, compiles on its own but cannot be placed as a block; use it through **RUNCHILD** instead.

7.3 Reasoning about reversibility

A sequence of reversible gates is reversible: undo the last gate first, then the one before it, and so on.

$$(U_k \cdots U_2 U_1)^{-1} = U_1^{-1} U_2^{-1} \cdots U_k^{-1}.$$

NorGate is OR followed by X. Each gate only XORs a value into Out, so the two gates commute and NOR undoes itself: placing the block twice in a row returns every wire to where it started. Two self-inverse gates in sequence are not self-inverse in general; the inverse of $U_2 U_1$ is $U_1 U_2$. Written as a rule, the block performs

$$\text{Out}' = \text{Out} \oplus \neg(A \vee B),$$

the same “target XOR function” shape as AND. That shape is always reversible, because applying it again XORs the same value back out.

Three things that make a custom gate irreversible

MEASURE inside the process The block collapses its wires every time it runs.

SET on the output `SET Out 0p` makes the compiler insert a hidden RESET, so the block *overwrites* the target ($\text{Out}' = f(A, B)$) instead of XORing into it. Whatever the target held before is erased. `NorGate` deliberately has no SET: a freshly compiled wire already starts at 0, so it passes the same truth table.

Temporary wires left dirty Helper tokens become hidden workspace wires. If a helper is not returned to 0 (uncomputed), it stays entangled with the inputs, and the visible wires alone no longer behave reversibly.

A quick test you can run in the builder

Place your block twice on the same wires and choose **Run all**. For a reversible out-of-place gate, every wire ends exactly as it started. Try it with the target's *name* start set to `0p` and then `1p`. If the target does not return, look for one of the three problems above.

7.4 Using a custom gate in protocol text and IF / ELSE

Once registered, a custom gate is also an ordinary protocol instruction. Its `-I` wires bind the process's **PARAMS** in order and its `-O` wires bind its **RETURNVALS**; the counts must match. Add `dg` or `inv` (as in `NORDg`) to run a reversible custom gate backwards.

```
NOR -I A B -O Out
```

This means custom gates work anywhere an IF / ELSE chain uses gates:

- **In a branch body.** The whole block runs or is skipped as one step:

```
MEASURE -I A
IF A=1
  NOR -I B C -O Out
ELSE
  X -I Out -O Out
ENDIF
```

Use this instead of `RUNCHILD`, which is not allowed inside an IF block because the child's gates would not inherit the condition.

- **As the test.** A custom gate with exactly one **RETURNVALS** can be the gate expression. It runs on a scratch copy of the state and its output wire is compared with 0, 1, or S, as described in Gate-expression conditions:

```
IF (NOR -I A B -O Out) = 1p
  X -I C -O C
ENDIF
```

Unlike the built-in AND, a custom gate always keeps its own wiring: its output is the `-O` wire you name, never a hidden scratch wire.

Custom gates are stored in the browser session, so a protocol that uses one compiles only after that gate is registered. A custom gate whose process uses itself is rejected when it runs.

7.5 Rules to remember

- The gate id must start with a letter, use only letters, digits, and underscores, and cannot match a built-in gate such as AND.
- Custom gates are saved for the current browser session only, not in the bundled catalog. Keep the `.qpucir` file to recreate one later. A protocol that names a custom gate needs it registered before compiling.
- A custom gate can wrap a process that itself uses `RUNCHILD`; the child processes are captured when you register it.

8 Truth-table companion syntax

Truth tables are metadata paired to a process; they are not `.qpucir` commands. A protocol describes *how* a circuit works; its companion `.qpuio` describes selected input/output expectations.

8.1 Text `.qpuio` structure

A text table may use spaces or commas:

```
MAIN-PROCESS: HalfAdder
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
0 0p 0p 0p 0p
1 0p 1p 0p 1p
2 1p 0p 0p 1p
3 1p 1p 1p 0p
```

8.2 MAIN-PROCESS: table header

The colon is required in the table format:

```
MAIN-PROCESS: HalfAdder
```

This name associates metadata with its protocol. Notice the deliberate syntax difference: protocol source writes `MAIN-PROCESS HalfAdder`, while the truth-table header writes `MAIN-PROCESS: HalfAdder`.

8.3 INPUTS: and OUTPUTS: column groups

```
INPUTS: A B
OUTPUTS: Carry Sum
```

These lines divide the columns into values supplied before simulation and values checked after simulation. Their order must match the later column header. `INPUTS` should correspond to state-typed `PARAMS`; `OUTPUTS` should correspond to `RETURNVALS`. `OUTPUTS` must contain at least one name.

Explicit groups remove ambiguity. If they are absent, the loader can derive the split only when a matching protocol is available and the combined `PARAMS` and `RETURNVALS` widths match the table header.

8.4 Column header row

```
# A B Carry Sum
```

The first nonmetadata row beginning with a hash and whitespace or comma is the ordered column header. Here A and B are inputs, followed by Carry and Sum outputs. Unlike an ordinary protocol hash comment, this row has structural meaning inside a `.qpuio` file.

8.5 Indexed data rows

```
0 0p 0p 0p 0p
1 0p 1p 0p 1p
```

The first cell is a decimal row index. Indices begin at zero and increase by one without gaps. Remaining cells align left-to-right with the column header. Every row must have exactly the expected width, and every state cell must be `0p`, `1p`, or `sp`.

For n binary input columns, a complete table contains 2^n rows. The usual ordering interprets input columns most-significant first: for two inputs the sequence is 00, 01, 10, 11. The loader checks sequential row indices, while semantic validation also rejects duplicate input patterns.

8.6 CSV spelling

Commas may replace whitespace consistently:

```
MAIN-PROCESS: HalfAdder
INPUTS: A,B
OUTPUTS: Carry,Sum
#,A,B,Carry,Sum
0,0p,0p,0p,0p
```

Spaces around commas are trimmed. Do not mix extra prose into a data row; trailing hash comments are allowed only where the table loader can strip them after the data content.

8.7 Partial truth tables

A partial table lists selected cases rather than all 2^n combinations. It must contain at least one row, no duplicate input pattern, and no more than 2^n rows. Partial tables are useful for state-machine transitions or focused regressions, but passing one proves behavior only for the listed cases.

Input count n	Complete row count 2^n	Example
0	1	constant-output process
1	2	0, 1
2	4	00, 01, 10, 11
3	8	000 through 111
4	16	0000 through 1111

8.8 JSON .qpuio envelope

The equivalent machine-oriented JSON form is:

```
{
  "format": "qpuio",
  "version": 1,
  "processName": "HalfAdder",
  "inputColumns": ["A", "B"],
  "outputColumns": ["Carry", "Sum"],
  "rows": [
    ["0p", "0p", "0p", "0p"],
    ["0p", "1p", "0p", "1p"],
    ["1p", "0p", "0p", "1p"],
    ["1p", "1p", "1p", "0p"]
  ]
}
```

The `format` value must be `qpuio` and `version` must be 1. Column names must be nonempty strings, `outputColumns` must not be empty, and every nested row array must have exactly

$$\text{inputColumns.length} + \text{outputColumns.length}$$

valid cells.

8.9 How a table test runs

For each listed row, the runner:

1. maps input cells to state PARAMS by column name and order;
2. creates the circuit's start-state vector;
3. executes every compiled gate, including child expansions;
4. measures the declared RETURNVALS;
5. compares only those measured outputs with expected output cells.

An expected output **sp** accepts either measured bit. An input **sp** initializes an equal plus-state superposition. Because one probabilistic sample cannot establish a full distribution, tables are strongest for deterministic basis-state behavior.

8.10 Protected bundled tables

The bundled SingleBitFullAdder, TwoBitFullAdder, FourBitFullAdder, and PhaseDemo tables are canonical site assets. Do not rewrite them to make an experiment pass. Create a new uploaded or corrected process/table pair when testing different expectations. This keeps examples reproducible and prevents a protocol correction from silently changing its specification.

9 Complete worked protocol

```
PARAMS: A:state B:state Theta:float

MAIN-PROCESS BellAndLogic
CREATETOKEN -I Q0 Q1 AndOut

SET Q0 $A
SET Q1 $B
SET AndOut Op

# Quantum segment
H -I Q0 -O Q0
CNOT -I Q0 -O Q1
PHASE=pi/4 -I Q1 -O Q1

# Basis-state comparison output
AND -I A B -O AndOut

MEASURE -I Q0
MEASURE -I Q1
MEASURE -I AndOut
RETURNVALS Q0 Q1 AndOut
```

Starting from $|00\rangle$, the H-CNOT segment prepares

$$|00\rangle \xrightarrow{H \otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

The phase gate then produces

$$\frac{|00\rangle + e^{i\pi/4}|11\rangle}{\sqrt{2}}.$$

The two quantum measurements are correlated, while **AndOut** compares the basis inputs through $A \wedge B$. A deterministic truth table can specify the logic output, but repeated simulation is needed to characterize Bell-state measurement probabilities.

10 Error conditions and practical rules

1. Unknown opcodes fail with **Unknown command**. **RESET** is not a source opcode.
2. Every QPU gate except **MEASURE** requires **-I** and **-O**, then exact registry arity is checked.
3. **PHASE** requires a valid nonzero-denominator angle after **=**.
4. **SWAP** requires two matching input/output wires.
5. **RUNCHILD**/**CALL** fails if the named process is absent from the library. Unknown **DECLARECHILD** names fail on the declaration line.
6. **SET** fails when its positional value is missing.
7. **RETURNVALS** order is semantically significant for child bindings and truth-table columns.
8. Use the same input/output token for single-qubit gates.
9. Initialize derived-gate targets deliberately; their operation is **XOR** into the existing target, not destructive assignment.
10. **FREE** and **DELETETOKEN** end a name. They reuse a wire only from a known $|0\rangle$.

11 Troubleshooting protocols

The protocol editor's **Protocol checks** panel separates errors from warnings. Start with the first error because one malformed instruction can make later messages less useful.

11.1 Unknown instruction

Problem fragment

```
FLIP -I Q -O Q
```

Typical error: Unknown command: FLIP

Why: FLIP is not a QPU instruction. The intended bit-flip gate is X.

Corrected fragment

```
X -I Q -O Q
```

11.2 Missing gate output

Problem fragment

```
H -I Q
```

Typical error: H requires an **-O** output.

Why: a one-qubit gate names the same flowing wire as both input and mutated output.

Corrected fragment

```
H -I Q -O Q
```

11.3 Wrong number of controls

Problem fragment

```
CNOT -I A B -O Target
```

Why: CNOT has one control. A two-control bit flip is CCNOT.

Choose the correction that matches the intent

```
CNOT -I A -O Target # one control
```

```
CCNOT -I A B -O Target # two controls
```

11.4 Invalid phase angle

Problem fragment

```
PHASE=ninety -I Q -O Q
```

Why: the angle must be numeric radians, degrees ending in d, or pi notation.

Equivalent corrected fragments

```
PHASE=pi/2    -I Q -O Q
PHASE=90d     -I Q -O Q
PHASE=1.5708  -I Q -O Q
```

11.5 Unknown child process

Problem fragment

```
RUNCHILD Adder -I A B Cin -O Carry Sum
```

Typical error: Unknown child process 'Adder'

Why: child source must exist in the process catalog and its MAIN-PROCESS name must match exactly. Load the child protocol first, or change Adder to the catalog's declared name.

11.6 Truth-table columns do not match

If protocol source declares

```
PARAMS: A:state B:state
...
RETURNVALS Carry Sum
```

then companion metadata must keep exactly that input and output order:

```
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
```

Renaming, reordering, or omitting a column changes the interface and causes validation errors.

11.7 Cycle-suffix warnings

An integer token suffix is checked against the process cycle. Q:1 while the process is still on cycle 0 produces CYCLE_SUFFIX_MISMATCH. Child processes count cycles from zero again. -\$R, JOIN, SPLIT, FREE, DELETETOKEN, COMPILEPROCESS, MASTERVAL, SAVE_STATE, and LOAD_STATE now execute; they are not metadata-only warnings. DECLARECHILD still binds a catalog child for later RUNCHILD or CALL.

11.8 Using the Correction Lab

1. In the protocol editor, read the reported line and suggestion.
2. Choose **Open Correction Lab for guided help**.
3. Confirm the selected process and its truth table. For a new experiment, use an uploaded or newly created process rather than modifying a protected bundled table.
4. Run tests or probe outputs to identify the first behavioral mismatch.
5. Apply gate-level guidance to protocol source.

6. Return to the Circuit builder, compile again, and confirm that Protocol checks no longer reports an error.

The Correction Lab is appropriate for syntax failures, child-process binding problems, and circuits whose measured outputs disagree with their truth table. It must not rewrite canonical bundled truth-table expectations.

12 Diagnostic message reference

Code	Meaning	First response
EMPTY_PROTOCOL	No instructions were found.	Add a complete protocol starting with MAIN-PROCESS.
EMPTY_PARAMS	PARAMS has no name:type entries.	Add an entry such as Input:state or remove PARAMS.
MISPLACED_PARAMS	PARAMS is not the first instruction.	Move it before MAIN-PROCESS.
UNKNOWN_INSTRUCTION	The first word is not supported.	Check spelling and the language reference.
INVALID_PHASE	PHASE has an unreadable angle.	Use pi/2, 90d, or numeric radians.
INVALID_GATE_ARITY	Input/control or output count is wrong.	Compare the gate's documented -I/-O counts.
COMPILE_ERROR	Syntax was readable but compilation could not complete.	Inspect child names, SET values, and SWAP bindings.
CYCLE_SUFFIX_MISMATCH	An integer : suffix does not match the process cycle.	Move the reference or add INCREASECYCLE.
INACTIVE_INVERSE_MARKER	A dg or inv marker on a gate other than PHASE, S, T, or a self-inverse primitive does not invert it.	Remove the marker from MEASURE.
MISSING_MAIN_PROCESS	InlineProcess fallback will be used.	Add a stable MAIN-PROCESS name.
MISSING_RETURN_VALUES	No public outputs are declared.	Add RETURNVALS in display order.

13 Glossary

Term	Meaning
Amplitude	A complex coefficient of a basis state. Its squared magnitude contributes to measurement probability.
Ancilla	An auxiliary qubit or workspace wire, usually prepared in a known state, used to hold intermediate information in a reversible computation.
Basis state	A state used as a coordinate direction. The computational basis for one qubit is $ 0\rangle, 1\rangle$.
Bloch sphere	A geometric representation of pure one-qubit states after ignoring global phase. North/south are $ 0\rangle/ 1\rangle$; longitude represents relative phase.
Boolean algebra	The mathematics of 0/1 values and operations such as NOT, AND, OR, and XOR.
Child process	A separately named protocol expanded into a parent through RUNCHILD or CALL.
Combinational logic	A classical circuit whose output depends only on current inputs, not stored history.

Term	Meaning
Compile-time recursion	Bounded self-expansion of a child process marked REC or TREC. RECUR expands another frame with DEPTH−1; EXIT WHEN ends a frame. Tail-position REC auto-converts to TCO (iterative one-frame expansion, F#-style); TREC requires that form. No runtime loop or recursive QPU execution is created.
IF / ELSE / ENDIF	Structured classical feed-forward sugar. Compiles to flat gates carrying −IF conditions (ELSE gates get the opposite value) and branch metadata for the canvas labels. The header is either a measured bit (IF A=1) or a gate expression evaluated on a scratch copy, such as IF (AND -I A B -O B) = 1p. Both branches stay in the circuit; the one whose test fails is skipped and drawn faded as “skipped”. Never forks the quantum wire.
Control	A wire whose basis condition decides whether a controlled operation acts on its target. The control itself is preserved by the standard controlled gates.
DEPTH	Read-only remaining recursive expansions for the current child frame. Supplied by RUNCHILD -DEPTH and decremented by RECUR.
Dn badge	Teal label above a recursive call on the canvas showing the current DEPTH; counts down while stepping and hides when the call ends.
Entanglement	A joint multi-qubit state that cannot be written as a tensor product of independent qubit states.
Fan-out	Sending one classical value to multiple destinations. Unknown quantum states cannot be universally copied; CNOT copies only known basis information onto a zero target and otherwise may entangle.
Gate	A state transformation. Closed-system quantum gates are unitary; measurement and reset have different semantics.
Garbage	Reversible-workspace information no longer wanted as output but still present because it cannot simply be erased.
Global phase	A unit complex multiplier applied to the whole state. It does not change physical measurement predictions.
Ket	Dirac notation $ \psi\rangle$ for a state column vector.
Measurement	Conversion of quantum amplitudes into a sampled classical result, with state collapse and renormalization.
Phase	The angular part of a complex amplitude. Relative phase controls interference even when immediate basis probabilities are unchanged.
Process	A named QPU protocol body with ordered PARAMS and RETURNVALS.
Protocol	Plain-text QPU instructions describing preparation, gates, composition, measurement, and returned references.
Qubit	A normalized two-component complex state $\alpha 0\rangle + \beta 1\rangle$, not merely an unknown classical bit.
Register	One or more logical wires considered together. Current JOIN/SPLIT syntax is accepted but does not yet lower to register operations.
Relative phase	The phase difference between amplitudes. Unlike global phase, it can change later interference and measurement probabilities.
Superposition	A coherent linear combination of basis states. sp prepares the equal plus superposition.
Target	The wire whose amplitudes a gate mutates; normally the first −0 reference.
Tensor product	The operation $\otimes$ that combines state spaces. Two one-qubit vectors produce a four-amplitude two-qubit vector.
Token	A source-level logical name resolved to a qubit wire or alias during compilation.
Truth table	Listed input/output expectations for measured RETURNVALS. It describes basis behavior but not arbitrary phase or entanglement.
Uncomputation	Running suitable inverse gates in reverse order to return temporary workspace to a known state without irreversible erasure.
Unitary	A matrix satisfying $U^\dagger U = I$, preserving normalization and having a reversible inverse.